Normalization and residual connections

Last modified: 2026-10-29

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; this page covers the part of his module on optimization and regularization[1] that deals with normalization and residual connections[2]. The exercises are new.

1 Welcome

1.1 Plan for this lecture

The backpropagation chapter found that the gradient[3] of a deep MLP can shrink layer by layer, and the optimization chapter chose starting weights to keep each layer's values at a steady size. This lecture tries two more ways to train a network of 20 such layers:

1. rescaling each layer's values during training, across the images of a minibatch or across the units of one image;
2. adding each layer's input to its output.

1.2 The digits and a deep network

```
import numpy as np
import pandas as pd
import torch
from torch import nn

_ = torch.manual_seed(481)
digits = pd.read_csv("data/optdigits-test.csv", header=None)
x = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
y = torch.tensor(digits[64].to_numpy())
n_train = 1200
order = torch.randperm(len(x))
x_train, y_train = x[order[:n_train]], y[order[:n_train]]
x_test, y_test = x[order[n_train:]], y[order[n_train:]]
loss_fn = nn.CrossEntropyLoss()


class Block(nn.Module):
    # One hidden layer of `width` ReLU units.
```

[1] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-regularization
[2] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-residual-connection
[3] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

```
    # make_norm(width) builds a layer applied to its pre-activations;
    # nn.Identity leaves them unchanged.

    def __init__(self, width, make_norm=nn.Identity, residual=False):
        super().__init__()
        self.linear = nn.Linear(width, width)
        self.norm = make_norm(width)
        self.residual = residual

    def forward(self, h):
        out = torch.relu(self.norm(self.linear(h)))
        if self.residual:
            return h + out
        return out


def make_deep(make_norm=nn.Identity, residual=False, depth=20, width=64, seed=1):
    # A first layer from the 64 pixels, `depth` blocks, then 10 logits.
    torch.manual_seed(seed)
    blocks = [Block(width, make_norm, residual) for _ in range(depth)]
    return nn.Sequential(nn.Linear(64, width), nn.ReLU(), *blocks, nn.Linear(width, 10))


def train(net, n_epochs=5, lr=0.01, batch_size=32, seed=0):
    # Minibatch SGD; returns the training loss and the test accuracy at the end.
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    generator = torch.Generator().manual_seed(seed)
    for epoch in range(n_epochs):
        net.train()
        shuffled = torch.randperm(n_train, generator=generator)
        for start in range(0, n_train, batch_size):
            batch = shuffled[start:start + batch_size]
            optimizer.zero_grad()
            loss_fn(net(x_train[batch]), y_train[batch]).backward()
            optimizer.step()
    net.eval()
    with torch.no_grad():
        train_loss = loss_fn(net(x_train), y_train).item()
        test_acc = (net(x_test).argmax(dim=1) == y_test).float().mean().item()
    return train_loss, test_acc


def first_layer_grad(net):
    # Size of the loss gradient for the first layer's weights, on 256 training images.
    net.train()
    net.zero_grad()
    loss_fn(net(x_train[:256]), y_train[:256]).backward()
    return net[0].weight.grad.norm().item()


def last_block_std(net):
    # Standard deviation of the last block's output, on 256 training images.
    net.train()
    with torch.no_grad():
        return net[:-1](x_train[:256]).std().item()
```

```
chance_loss = np.log(10)

print(make_deep()[:3])
```

```
Sequential(
  (0): Linear(in_features=64, out_features=64, bias=True)
  (1): ReLU()
  (2): Block(
    (linear): Linear(in_features=64, out_features=64, bias=True)
    (norm): Identity()
  )
)
```

The exercises use the digits split of the feedforward networks chapter, with 1,200 training images. `make_deep()` builds an MLP with 21 hidden layers of 64 ReLU units: a first layer from the pixels, then 20 copies of `Block`. `train()` runs 5 epochs[4] of minibatch stochastic gradient descent[5], with batch size 32 and learning rate[6] 0.01. `first_layer_grad()` measures the size of the gradient[7] that reaches the first layer. The exercises count a network as having learned something when its training loss after `train()` is below 90% of $\log 10 \approx 2.303$, the loss of predicting $\frac{1}{10}$ for every digit.

2 A deep network

2.1 Training 20 layers

Exercise 2.1 (A 20-block network)**.** Does `make_deep()` learn anything in the 5 epochs[a] of `train()`?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

Solution 2.1.

```
plain_grad = first_layer_grad(make_deep())
plain_loss, plain_acc = train(make_deep())
plain_std = last_block_std(make_deep())
print(plain_grad, plain_loss, plain_acc, plain_std)
```

```
1.927740855833804e-09 2.3031463623046875 0.1072026789188385 0.035105034708976746
```

No. Its training loss after 5 epochs[a] is 2.303, against $\log 10 \approx 2.303$ for probabilities of $\frac{1}{10}$. It classifies 10.7% of the test images correctly. The gradient[b] that reaches its first layer has size 1.9e-09. The last block's output has standard deviation 3.5e-02. So the gradient vanishes before it reaches the first layer, as in the backpropagation chapter.

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

[4] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch
[5] https://morrison-lab.github.io/mds/optimization.html#def-gradient-descent
[6] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-learning-rate
[7] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

3 Batch normalization

3.1 Definition

Definition 3.1 (Batch normalization)**. Batch normalization** rescales each pre-activation of a layer across the examples of a minibatch. For a minibatch of B examples, let z_{ij} be pre-activation j for example i, with mean and variance over the minibatch

$$\mu_j = \frac{1}{B}\sum_{i=1}^{B} z_{ij}, \qquad \sigma_j^2 = \frac{1}{B}\sum_{i=1}^{B}(z_{ij} - \mu_j)^2.$$

Batch normalization replaces z_{ij} with

$$\gamma_j \frac{z_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} + \beta_j,$$

where γ_j and β_j are parameters[a] learned with the rest and ε is a small constant that keeps the division finite [1], [2, Sec. 8.7.1]. When the network makes predictions, running averages of μ_j and σ_j^2 from training take the place of the minibatch's. PyTorch's running average of the variance divides by $B - 1$ instead of B [3].

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Example 3.1 (`nn.BatchNorm1d` on a minibatch)**.**

```
batch_norm = nn.BatchNorm1d(64)
pre = nn.Linear(64, 64)(x_train[:32]).detach()
normed = batch_norm(pre).detach()
print(pre.mean(dim=0)[:4], pre.std(dim=0, correction=0)[:4])
```

```
tensor([ 0.1275, -0.3611,  0.5323,  0.4476]) tensor([0.1708, 0.1277, 0.1898, 0.1267])
```

```
print(normed.mean(dim=0)[:4], normed.std(dim=0, correction=0)[:4])
```

```
tensor([6.7055e-08, 2.5332e-07, 4.4703e-08, 1.8626e-07]) tensor([0.9998, 0.9997, 0.9999, 0.9997])
```

```
largest_mean = normed.mean(dim=0).abs().max().item()
std_min = normed.std(dim=0, correction=0).min().item()
std_max = normed.std(dim=0, correction=0).max().item()
```

`nn.BatchNorm1d(64)` applies batch normalization to 64 pre-activations. It starts with every $\gamma_j = 1$ and every $\beta_j = 0$ [3]. For 32 training images, the first 4 pre-activations of a random layer have the means and standard deviations on the first line. After batch normalization, on the second line, no column's mean is farther from 0 than 3.3e-07, and the standard deviations run from 0.999 to 1.000.

3.2 A batch of one

Exercise 3.1 (Predicting for one image)**.** Why can batch normalization not use the mean and variance of the minibatch when a network makes a prediction for a single image?

Solution

Solution 3.1. A minibatch of one image has $\mu_j = z_{1j}$ and $\sigma_j^2 = 0$, so every normalized pre-activation would be 0, whatever the image. The prediction would not depend on the image at all.

```
try:
    _ = batch_norm(pre[:1])
except ValueError as error:
    batch_of_one_error = str(error)
print(batch_of_one_error)
```

```
Expected more than 1 value per channel when training, got input size torch.Size([1, 64])
```

```
_ = batch_norm.eval()
one_image_output = batch_norm(pre[:1])
print(one_image_output.shape)
```

```
torch.Size([1, 64])
```

`nn.BatchNorm1d` treats a minibatch of one differently in its two modes:

- **Training mode.** It refuses the minibatch: "Expected more than 1 value per channel when training, got input size torch.Size([1, 64])".
- **Evaluation mode.** It uses its running averages instead, so it accepts a single image.

Evaluation mode is what makes predictions use the running averages. That is why `train()` calls `net.eval()` before it computes the test accuracy[a].

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

3.3 Batch normalization in the deep network

Exercise 3.2 (Batch normalization in the 20-block network)**.** Does `make_deep(make_norm=nn.BatchNorm1d)` learn in the 5 epochs[a] of `train()`?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

Solution 3.2.

```
batch_grad = first_layer_grad(make_deep(make_norm=nn.BatchNorm1d))
batch_loss, batch_acc = train(make_deep(make_norm=nn.BatchNorm1d))
batch_std = last_block_std(make_deep(make_norm=nn.BatchNorm1d))
print(batch_grad, batch_loss, batch_acc, batch_std)
```

```
20.3425235748291 1.379925012588501 0.6683416962623596 0.6066936254501343
```

Yes. With batch normalization in every block, the training loss after 5 epochs[a] is 1.380. The network classifies 66.8% of the test images correctly. The gradient[b] at the first layer has size 2.0e+01, against 1.9e-09 without it. Each block rescales its pre-activations to standard deviation 1 before the ReLU, so the values cannot shrink from block to block: the last block's output has standard deviation 0.607, against 3.5e-02 without batch normalization.

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch
[b] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

4 Layer normalization

4.1 Definition

Definition 4.1 (Layer normalization)**. Layer normalization** rescales the pre-activations of a layer across the units of one example. For an example with pre-activations $z_1, \dots, z_L$, it uses their mean and variance

$$\mu = \frac{1}{L}\sum_{j=1}^{L} z_j, \qquad \sigma^2 = \frac{1}{L}\sum_{j=1}^{L}(z_j - \mu)^2,$$

and replaces each z_j with $\gamma_j(z_j - \mu)/\sqrt{\sigma^2 + \varepsilon} + \beta_j$ [4]. It does the same in training and in prediction, since it never looks at other examples.

Example 4.1 (`nn.LayerNorm` on one image)**.**

```
layer_norm = nn.LayerNorm(64)
row_normed = layer_norm(pre[:1]).detach()
print(row_normed.mean().item(), row_normed.std(correction=0).item())
```

```
-2.0489096641540527e-08 0.9999433159828186
```

`nn.LayerNorm(64)` applies layer normalization to each row of its input [5]. For the first training image alone, its 64 outputs have mean -2.0e-08 and standard deviation 1.000.

4.2 Layer normalization in the deep network

Exercise 4.1 (Layer normalization in the 20-block network)**.** Does `make_deep(make_norm=nn.LayerNorm)` learn in the 5 epochs[a] of `train()`?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

Solution 4.1.

```
layer_grad = first_layer_grad(make_deep(make_norm=nn.LayerNorm))
layer_loss, layer_acc = train(make_deep(make_norm=nn.LayerNorm))
print(layer_grad, layer_loss, layer_acc)
```

```
0.39439767599105835 2.3055310249328613 0.09380234777927399
```

No. With layer normalization, the training loss after 5 epochs[a] is 2.306, and the test accuracy[b] is 9.4%, against 1.380 and 66.8% with batch normalization. The gradient[c] at the first layer has size 3.9e-01. Here layer normalization learns less than batch normalization in the same 5 epochs. Because layer normalization never uses other examples, it also works for a minibatch of one and for sequences of different lengths [4].

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch
[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[c] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

5 Residual connections

5.1 A residual block

Remark (Residual connections in `Block`)

Remark 5.1 (Residual connections in `Block`). `Block(width, residual=True)` returns $h + \mathrm{ReLU}(\mathbf{W}h + b)$ for its input h, a residual connection[a] around one hidden layer. This block is simpler than those of He et al. [6], which go around two or three convolutional layers and apply the ReLU after the sum. The input and output have the same width[b], 64, so the sum is defined.

[a] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-residual-connection
[b] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-width

5.2 Gradients with residual connections

Exercise 5.1 (The gradient through residual connections)**.** How large is the gradient[a] at the first layer of `make_deep(residual=True)`, compared with `make_deep()`?

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Solution

Solution 5.1.

```
residual_grad = first_layer_grad(make_deep(residual=True))
print(residual_grad, plain_grad)
```

```
25.32086181640625 1.927740855833804e-09
```

The gradient[a] has size 2.5e+01, against 1.9e-09 without residual connections[b]. A block computes $h + f(h)$, with $f(h) = \mathrm{ReLU}(\mathbf{W}h + b)$ from Remark 5.1, so its derivative with respect to h is the identity matrix plus the derivative of f. Going back through a block, the gradient keeps a copy of itself through the identity. It keeps that copy however small the derivative of f is. Without the residual connection[c], the gradient passes only through the derivative of f, and shrinks when that derivative is small. Across 20 blocks these derivatives multiply. So when each factor is below 1 in size, the gradient shrinks geometrically.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient
[b] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-residual-connection
[c] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-residual-connection

5.3 Four networks

Exercise 5.2 (Four 20-block networks)**.** Which of `make_deep()`, `make_deep(make_norm=nn.BatchNorm1d)`, `make_deep(residual=True)` and `make_deep(make_norm=nn.BatchNorm1d, residual=True)` reaches the lowest training loss in the 5 epochs[a] of `train()`?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-epoch

Solution

Solution 5.2.

```
settings = {
    "no normalization or residual connections": {},
    "batch normalization": {"make_norm": nn.BatchNorm1d},
    "residual connections": {"residual": True},
    "batch normalization and residual connections": {"make_norm": nn.BatchNorm1d, "residual": True},
}
four_rows = []
for name, options in settings.items():
    loss, acc = train(make_deep(**options))
    four_rows.append({"network": name, "train_loss": loss, "test_accuracy": acc})
four_table = pd.DataFrame(four_rows)
best_network = four_table.loc[four_table["train_loss"].idxmin()]
residual_only_loss = four_table.set_index("network").loc["residual connections", "train_loss"]
print(four_table.round(3))
```

```
                                       network  train_loss  test_accuracy
0      no normalization or residual connections       2.303          0.107
1                           batch normalization       1.380          0.668
```

```
2                          residual connections       1.434          0.561
3  batch normalization and residual connections       0.035          0.953
```

The network with batch normalization and residual connections reaches the lowest training loss, 0.035, and classifies 95.3% of the test images correctly. The residual network without batch normalization reaches 1.434. The residual networks of He et al. [6] also put batch normalization inside each block.

5.4 Further reading

Read

Deep Learning's section 8.7.1[a] covers batch normalization [2]. Ioffe and Szegedy [1] introduced batch normalization, Ba et al. [4] layer normalization, and He et al. [6] residual networks.

[a] https://www.deeplearningbook.org/contents/optimization.html

6 References

[1] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd international conference on machine learning*, in Proceedings of machine learning research, vol. 37. 2015, pp. 448–456. Available: https://proceedings.mlr.press/v37/ioffe15.html

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[3] PyTorch Contributors, "Torch.nn.BatchNorm1d." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html

[4] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization." 2016. Available: https://arxiv.org/abs/1607.06450

[5] PyTorch Contributors, "Torch.nn.LayerNorm." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90[8].

[8] https://doi.org/10.1109/CVPR.2016.90